

[Diagrama de Clases de Implementación: Gestión de Biblioteca Universitaria]

Estudiante: Nicolás Monfort Albi

C.I.: 6706591-2

Matrícula/ID: 0000581635

Nombre de la asignatura: Programación 2

Fecha de entrega: 13 de Mayo de 2026

1. Identificación de Clases Principales

En esta etapa se determinan las entidades que representan los conceptos clave del dominio de la biblioteca:

Libro: Representa el recurso físico o digital disponible.

Usuario: Representa a la persona que interactúa con la biblioteca.

Préstamo: Entidad que gestiona la transacción y el vínculo temporal entre un usuario y un libro.

Categoría: Entidad para organizar y clasificar el catálogo.

2. Definición de Atributos

Para cada clase, se definen los datos necesarios para su correcta gestión:

Libro:

Título

Autor

ISBN

Estado (Disponible/Prestado).

Usuario:

ID de Usuario

Nombre

Dirección

Teléfono.

Préstamo:

Fecha de inicio

Fecha de devolución prevista

Fecha de devolución real.

Categoría:

Nombre de la categoría

Código identificador.

3. Establecimiento de Relaciones

Categorización: Un Libro pertenece a una Categoría (Relación 1:N).

Registro de Préstamo: Un Usuario puede tener muchos Préstamos, pero cada Préstamo está asociado a un único Usuario y a un único Libro en un momento dado.

Restricción de disponibilidad: Se define que un Libro solo puede estar vinculado a un Préstamo activo si su atributo estado es "Disponible".

4. Inclusión de Métodos

Para dotar de comportamiento a las entidades y permitir la interacción entre ellas, se han definido los siguientes métodos principales:

Clase Usuario:

solicitarPrestamo(libro: Libro): Inicia el proceso de préstamo de un ejemplar específico.

consultarDisponibilidad(libro: Libro): Permite al usuario verificar si un título está apto para ser retirado.

Clase Libro:

actualizarEstado(nuevoEstado: String): Cambia la condición del libro (ej. de "Disponible" a "Prestado") tras una operación exitosa.

obtenerInformacion(): Devuelve los detalles bibliográficos del ejemplar.

Clase Préstamo:

registrarDevolucion(): Finaliza el ciclo del préstamo, registrando la fecha real de entrega.

verificarVencimiento(): Compara la fecha actual con la de devolución prevista para determinar si el préstamo está fuera de plazo.

Clase Categoría:

listarLibrosAsociados(): Genera una lista de todos los libros que pertenecen a dicha clasificación.

5. Aplicación de Restricciones

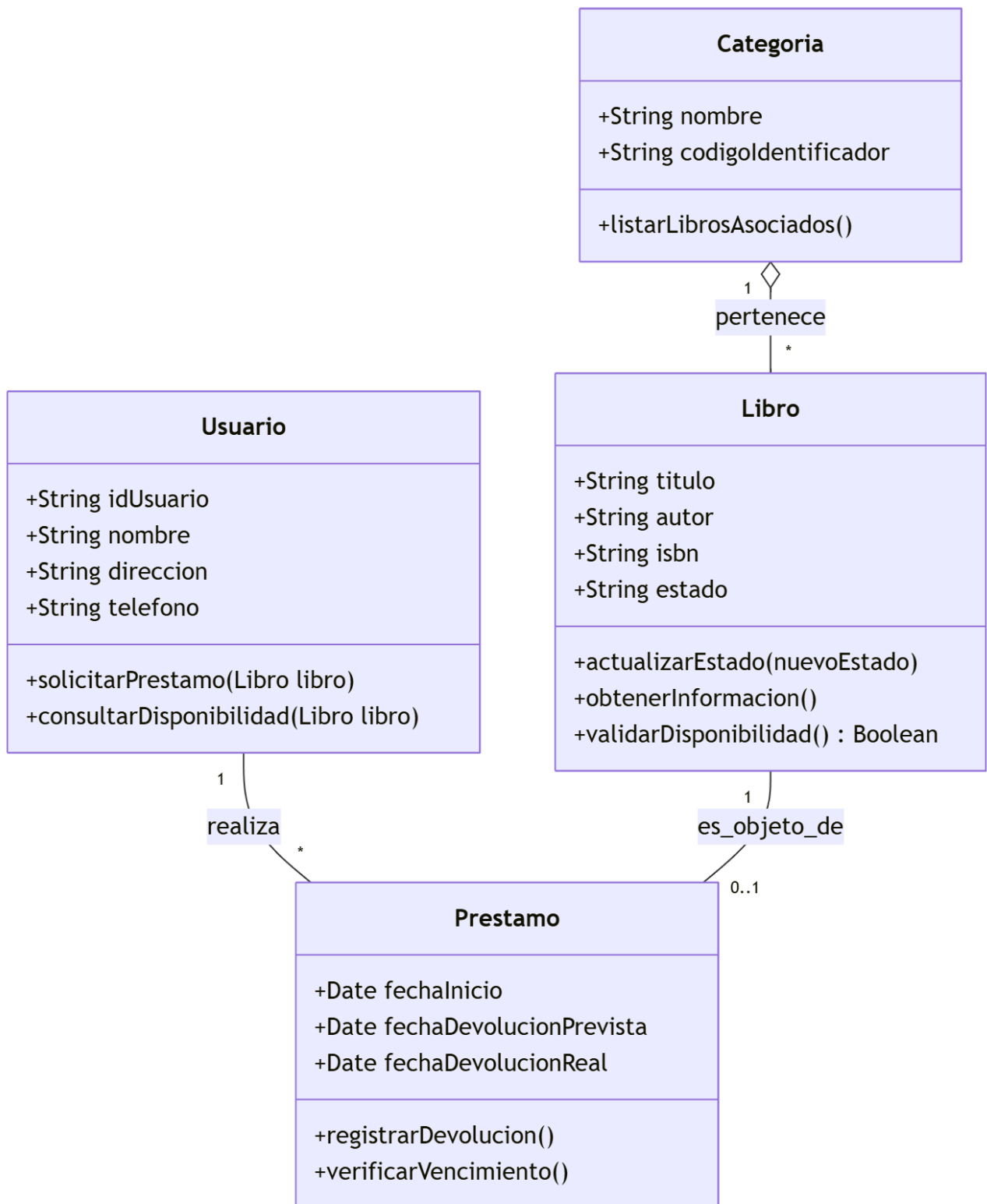
Para asegurar la integridad de los datos y el cumplimiento de las reglas del negocio, se han definido y representado las siguientes restricciones:

Restricción de Disponibilidad: No se permite la creación de un objeto Préstamo si el atributo estado de la clase Libro es diferente a "Disponible". Esta regla garantiza que los recursos no operativos o ya retirados no puedan ser asignados nuevamente.

Prevención de Préstamos Duplicados: Mediante la lógica de cardinalidad y estado, se asegura que un Libro solo pueda estar asociado a un único Préstamo activo. Una vez que se registra un préstamo, el estado del libro cambia automáticamente, bloqueando cualquier intento de duplicación de la transacción para el mismo ejemplar.

Validación de Identidad: El sistema exige que todo Préstamo esté vinculado obligatoriamente a un idUsuario y un ISBN válidos, evitando registros huérfanos o inconsistentes.

6. Diagrama de Clases



7. Descripción del Diagrama de Clases

El diseño propuesto emplea una arquitectura modular centrada en la trazabilidad del ejemplar. Se han identificado cuatro clases núcleo: Libro, Usuario, Categoría y Préstamo. La decisión clave de diseño fue utilizar la clase Préstamo como el eje que vincula la entidad Libro con Usuario, permitiendo un registro histórico independiente.

Las relaciones se definieron con multiplicidades estrictas para asegurar la integridad: un libro solo puede pertenecer a un préstamo activo, validado mediante el atributo de estado y métodos de comprobación de disponibilidad, cumpliendo así con las restricciones de evitar préstamos duplicados.

Aunque se consideró la inclusión de una clase Sanción para gestionar devoluciones tardías o daños (tal como se discutió en clase), se ha decidido mantener este modelo como una extensión lógica para una fase posterior de desarrollo, priorizando en esta entrega la robustez del flujo principal de préstamo y devolución. Este enfoque garantiza un modelo limpio, escalable y alineado con los requisitos funcionales solicitados.

